
Machine Learning Fundamentals

A Theory Companion to Part I

the landscape · end-to-end projects · classification · training models · SVMs · trees ·
ensembles · dimensionality reduction · unsupervised learning

Computing, Robotics & Bionics

3rd-year Bioengineering — a study companion aligned to Part I of A. Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*

*This document is an **original theory companion** to Part I of Géron's book (the classic, non-neural machine-learning fundamentals, Chapters 1–8, plus an online appendix on SVMs). It is written from scratch in our own words and notation, developed with fuller derivations than a hands-on text has room for, and illustrated with biomedical examples. It is not a reproduction of the book: it contains none of the author's text, code or figures, and is meant to be read alongside the book and its notebooks — the book supplies the worked code and datasets; this companion supplies the mathematics and the “why”. It assumes the three refresher modules (Python/NumPy/Pandas, linear algebra, calculus): you will see the normal equation reappear as the projection of Module 2, gradient descent as the optimisation of Module 3, and the dot product as the engine of every linear model. Each chapter states the model, derives its training objective and the rule that optimises it, and notes its assumptions and failure modes.*

Coverage: Chapters 1–8 plus the SVM appendix (Appendix C in the notebooks) — Part I complete.
Prerequisites: Refresher Modules 1–3. Read with the book and the `ageron/handson-mlp` notebooks open.

Contents

1	The Machine Learning Landscape	2
1.1	A taxonomy of learning	2
1.2	Overfitting, underfitting and capacity	2
1.3	Estimating generalisation honestly	2
2	End-to-End Machine Learning Project	3
2.1	Performance measures: which norm?	4
2.2	Building a trustworthy test set	4
2.3	Preparing data	4
2.4	Selecting a model	4
3	Classification	5
3.1	The confusion matrix and its ratios	5
3.2	The threshold, and the precision–recall trade-off	6
3.3	ROC, AUC, and rare positives	6
3.4	Beyond two classes	6
4	Training Models	7
4.1	Linear regression and the normal equation	7
4.2	Gradient descent	7
4.3	Polynomial regression, learning curves, and the trade-off	8
4.4	Regularised linear models	8
4.5	Logistic regression	8
4.6	Softmax regression	9
5	Decision Trees	10
5.1	Impurity	10
5.2	The CART training objective	10
5.3	Regularisation and the catch	11
6	Ensemble Learning and Random Forests	11
6.1	Why voting works	11
6.2	Bagging, pasting and out-of-bag	11
6.3	Random forests	12
6.4	Boosting	12
6.5	Stacking	12
7	Dimensionality Reduction	13
7.1	The curse of dimensionality	13
7.2	PCA via the SVD	13
7.3	Variants and manifold methods	14
8	Unsupervised Learning Techniques	14
8.1	k -means	14
8.2	DBSCAN	15
8.3	Gaussian mixtures and EM	15
A	Support Vector Machines	16
A.1	The margin	16
A.2	Soft margin and the hinge loss	16
A.3	The dual and the kernel trick	17

1 The Machine Learning Landscape

Machine learning is the science of building programs that improve at a task through *experience* (data) rather than through explicit rules. Formally, a program learns from experience E at task T measured by performance P if its P on T improves with E . The art is choosing a *model* (a parameterised family of functions), a *cost* that scores how badly it fits, and an *optimiser* that tunes the parameters to minimise that cost — the three ingredients we make precise from Chapter 4 onward.

1.1 A taxonomy of learning

- **Supervised** — each training example carries a label. If the label is continuous the task is *regression* (predict a tumour’s diameter); if discrete, *classification* (benign vs malignant).
- **Unsupervised** — no labels. Tasks include *clustering* (group patients by expression profile), *dimensionality reduction* (Chapter 7), *density estimation* and *anomaly detection* (flag an abnormal ECG).
- **Self-/semi-supervised** — create labels from the data itself, or mix a little labelled with much unlabelled data.
- **Reinforcement** — an agent learns a policy by trial and error against a reward signal (a closed loop, conceptually close to the Control unit).

Two orthogonal axes cut across this list. **Batch vs online**: a batch learner trains once on all data; an online learner updates incrementally as data streams in. **Instance- vs model-based**: an instance-based learner (e.g. k -nearest neighbours) memorises examples and generalises by similarity; a model-based learner fits parameters and generalises through the fitted function. This course is almost entirely model-based.

1.2 Overfitting, underfitting and capacity

A model has limited **capacity** (flexibility). Too little and it **underfits** — it cannot capture the signal, so both training and test error are high (high *bias*). Too much and it **overfits** — it memorises noise, so training error is low but test error is high (high *variance*). The cure for overfitting is more data, fewer features, or **regularisation**: constraining the parameters so the model cannot contort too freely (Chapter 4). Knobs that are set before training rather than learned — a regularisation strength, a tree depth — are **hyperparameters**, tuned on validation data.

1.3 Estimating generalisation honestly

We care about error on *unseen* data, so we hold out a **test set** and never touch it during development. Model and hyperparameter choices are made on a separate **validation set** (or by **k -fold cross-validation**, which rotates the validation fold k times and averages, using data efficiently). Tuning on the test set leaks information and inflates your estimate — a subtle but serious error.

The bias–variance decomposition Suppose the truth is $y = f(\mathbf{x}) + \varepsilon$ with noise $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$. Let $\hat{f}$ be our model trained on a random dataset. The expected squared error at a point $\mathbf{x}$, averaged over datasets and noise, splits into three non-negative pieces:

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}))^2] = \underbrace{(f(\mathbf{x}) - \mathbb{E}[\hat{f}(\mathbf{x})])^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{f}(\mathbf{x}))}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

Sketch. Write $y - \hat{f} = (y - f) + (f - \mathbb{E}\hat{f}) + (\mathbb{E}\hat{f} - \hat{f})$. Expand the square and take expectations; the cross terms vanish because ε is independent of $\hat{f}$ and $\mathbb{E}[\mathbb{E}\hat{f} - \hat{f}] = 0$. The first term is the noise σ^2 , the second the squared bias, the third the variance.

This decomposition is the theoretical backbone of the whole part: lowering bias (more capacity) tends to raise variance and vice versa, and good models trade them off. The irreducible σ^2 sets a floor no model can beat.

Key idea

No Free Lunch. Averaged over *all* possible problems, no learning algorithm beats any other; an algorithm is good only because its assumptions *match* the problems we actually face. So model selection is really assumption selection — which is why understanding each model’s assumptions (stated in every chapter below) matters more than memorising its API.

Exercises

1. Classify each task, and say what the training signal is: (a) predicting post-operative length of stay from pre-operative variables; (b) grouping gait recordings into patterns when no labels exist; (c) a prosthetic hand controller that improves from a grip-success reward; (d) learning a representation by predicting a masked part of an ECG trace from the rest.
2. Model A has training error 0.2 and validation error 2.4; model B has 2.3 and 2.4. Diagnose each, and give one fix for each that does *not* involve collecting more data.
3. Complete the proof sketched in the bias–variance box: expand $\mathbb{E}[(y - \hat{f})^2]$ with $y - \hat{f} = (y - f) + (f - \mathbb{E}\hat{f}) + (\mathbb{E}\hat{f} - \hat{f})$ and show each cross term vanishes, naming the independence or zero-mean fact you use.
4. The irreducible term σ^2 does not shrink with more data. Say what *would* reduce it in a real biomedical measurement, and why that is a laboratory question rather than a modelling one.
5. “No Free Lunch says all models are equally good.” Correct this in one sentence, and state what the theorem actually constrains.
6. You scale the features using the mean and standard deviation of the *whole* dataset and then split into train and validation. Name the quantity that is now optimistically biased, and explain the mechanism.

2 End-to-End Machine Learning Project

Before the algorithms, the *workflow*. A real project is: frame the problem, choose a performance measure, get and explore the data, build a clean test set, prepare the data, train and select a model, and validate. The modelling is a small slice; data handling dominates.

2.1 Performance measures: which norm?

For regression we summarise the error vector $\hat{\mathbf{y}} - \mathbf{y}$ over m examples by a norm (Module 2). Two are standard:

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2} \quad (\ell_2), \quad (2.1)$$

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i| \quad (\ell_1). \quad (2.2)$$

RMSE squares errors, so it **penalises large mistakes more heavily** and is the right measure when big errors are especially bad and outliers are few; MAE is more **robust** to outliers. The choice is not cosmetic — it changes which model “wins”, and (Chapter 4) which cost we minimise.

2.2 Building a trustworthy test set

Split off a test set *once*, early, and freeze it. Pure random splitting is fine for large data, but for a small or skewed dataset use **stratified sampling**: partition by an important attribute (say age band, or disease stage) and sample within each stratum, so the test set mirrors the population. Otherwise a sampling fluke can bias your estimate. Beware **data-snooping bias**: looking at the test set — even informally — lets patterns leak into your choices.

2.3 Preparing data

Models want clean numeric matrices (Module 1). The usual steps:

- **Impute** missing values (median for skewed clinical variables).
- **Encode** categoricals — *one-hot* for unordered categories (blood type), so the model does not invent a false ordering.
- **Scale** features, because gradient descent and distance-based methods are sensitive to scale. Two options:

$$\text{min-max: } x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1], \quad \text{standardise: } x' = \frac{x - \mu}{\sigma}.$$

Standardisation (zero mean, unit variance) is less sensitive to outliers and is the common default; it is exactly the centring-and-scaling of Modules 1–2.

Key idea

Fit transforms on the training set only, then apply the stored parameters to validation and test. Computing the mean/variance (or $x_{\min}, x_{\max}$) on the whole dataset leaks test information into training — the scaling version of data snooping. A *pipeline* chains these steps so the same fitted transformation is reused everywhere, preventing the leak.

2.4 Selecting a model

Train a few candidates, compare them by cross-validation, and only then estimate final performance *once* on the test set. If training error is high the model underfits (try more capacity or better features); if training error is low but validation error high it overfits (regularise, simplify, or get more data) — the bias–variance trade-off of Chapter 1, now operational. Hyperparameters are searched by grid or randomised search over cross-validated scores.

A biomedical framing

Predicting a continuous biomarker from 30 assay features: frame as regression, choose RMSE (large errors are clinically costly), stratify the test split by disease stage so all stages are represented, standardise the features on the training fold only, cross-validate a linear model against a tree ensemble, and report a single test RMSE with a rough confidence interval. Every later chapter slots a concrete model into this skeleton.

Exercises

1. RMSE and MAE are the ℓ_2 and ℓ_1 norms of the error vector (Module 2). A pulse-oximetry dataset contains a handful of gross sensor dropouts. Which measure would you report, and which would you *optimise*? They need not be the same answer — justify both choices.
2. An outcome occurs in 3% of samples and you take a random 20% test set from $m = 500$. Give the expected number of positives and a rough spread, and say why stratified sampling is not a refinement but a necessity here.
3. Each patient contributes several records. Explain why splitting rows at random leaks, what you must split on instead, and what the leaked model's test score would overstate.
4. You need to impute missing values, scale, and one-hot encode. Give a defensible order for the three, and state precisely which statistics may be estimated on which part of the data.
5. With $m = 300$ samples, argue for 5-fold cross-validation over a single hold-out validation split, and name the cost you pay for it.
6. A colleague tunes twenty hyperparameter settings on the test set and reports the best score. Name the effect, and say how large a gap you would expect between that score and a fresh test set.

3 Classification

In classification the label is a category. Start with the binary case (positive vs negative, e.g. malignant vs benign) and the right way to measure success — because raw **accuracy** lies on imbalanced data: a detector that calls *everything* negative scores 99% accuracy on a disease with 1% prevalence, while being useless.

3.1 The confusion matrix and its ratios

Every prediction falls into one of four cells: true positive (TP), false positive (FP), true negative (TN), false negative (FN). From these:

$$\text{precision} = \frac{TP}{TP + FP} \quad (\text{of those flagged positive, how many truly are}), \quad (3.1)$$

$$\text{recall (sensitivity)} = \frac{TP}{TP + FN} \quad (\text{of the true positives, how many we caught}), \quad (3.2)$$

$$\text{specificity} = \frac{TN}{TN + FP} \quad (\text{of the true negatives, how many we cleared}). \quad (3.3)$$

There is a tension: catching more positives (raising recall) usually flags more false alarms (lowering precision). The F_1 **score** combines them as a *harmonic* mean,

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \quad (3.4)$$

The harmonic mean is used (not the arithmetic) because it is dragged toward the *smaller* of the two: F_1 is high only when precision *and* recall are both high, so it punishes a model that sacrifices one for the other.

3.2 The threshold, and the precision–recall trade-off

A classifier really outputs a *score*; a **decision threshold** converts it to a label. Raising the threshold flags fewer cases — precision up, recall down; lowering it does the reverse. Plotting precision and recall against the threshold (or against each other) lets you pick the operating point your application needs.

Key idea

Which to favour? In disease screening a false negative (missed disease) is far costlier than a false alarm, so you tune for high **recall**, accepting lower precision and a confirmatory follow-up test. In a spam-style filter a false positive (a real message lost) is worse, so you favour **precision**. The metric must encode the clinical or operational cost; there is no universally “best” threshold.

3.3 ROC, AUC, and rare positives

The **ROC curve** plots the true-positive rate (recall) against the false-positive rate $FP/(FP + TN) = 1 - \text{specificity}$ as the threshold sweeps. A perfect model hugs the top-left; the diagonal is random guessing. The **area under the curve (AUC)** summarises this in one number (1.0 perfect, 0.5 chance) and equals the probability the model scores a random positive above a random negative. *Caveat:* when positives are very rare, the ROC can look optimistic; the **precision–recall curve** is then more informative because it ignores the easy, abundant true negatives.

3.4 Beyond two classes

For $K > 2$ classes, strategies built on binary classifiers are **one-vs-rest** (train K detectors, pick the highest score) and **one-vs-one** (train $\binom{K}{2}$ pairwise classifiers and vote); many algorithms also handle multiple classes natively (the softmax model of Chapter 4). **Multilabel** tasks assign several labels at once (which of many findings are present in a scan); **multioutput** tasks predict several multi-class targets together.

Exercises

1. A classifier on 1000 samples gives $TP = 45$, $FP = 15$, $FN = 5$, $TN = 935$. Compute accuracy, precision, recall, specificity and F_1 , and say which single number you would refuse to report alone.
2. For a screening test that triages patients for an expensive confirmatory scan, state which of precision and recall you would push up, which way you move the decision threshold, and what you pay for it.
3. Prevalence is 1% and the test has sensitivity 99% and specificity 99%. Compute the precision (positive predictive value) and explain the base-rate effect in one sentence.
4. With 2% positives, explain why the ROC curve can look excellent while the precision–recall curve looks poor, and which you would put in a paper.
5. AUC equals the probability that a randomly chosen positive is ranked above a randomly chosen negative. Explain why that makes AUC independent of the threshold — and

why that is both its strength and its blind spot.

6. For $K = 10$ classes, how many binary classifiers does one-versus-rest need, and how many one-versus-one? Say when the larger number is nevertheless cheaper to train.

4 Training Models

This chapter is the mathematical heart of Part I. We take linear models and derive, from scratch, both how to train them in closed form and how to train them by gradient descent (Module 3), then add regularisation, and finally extend linear regression to classification through logistic and softmax regression.

4.1 Linear regression and the normal equation

A linear model predicts a weighted sum of the features plus a bias,

$$\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n = \boldsymbol{\theta}^\top \mathbf{x}, \quad (4.1)$$

where we absorb the bias by setting $x_0 = 1$. Stacking the m training rows into a matrix $\mathbf{X}$ (the design matrix, one sample per row) and targets into $\mathbf{y}$, the **mean-squared-error** cost is

$$J(\boldsymbol{\theta}) = \frac{1}{m} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2. \quad (4.2)$$

This is a convex quadratic in $\boldsymbol{\theta}$, so its unique minimum is where the gradient vanishes. Using the matrix-calculus result $\nabla_{\boldsymbol{\theta}} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 = 2\mathbf{X}^\top(\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$,

$$\nabla J = \frac{2}{m} \mathbf{X}^\top(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) = \mathbf{0} \implies \mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y} \implies \boxed{\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}}. \quad (4.3)$$

This is the **normal equation**. Geometrically it is precisely the projection of Module 2: it forces the residual $\mathbf{X}\boldsymbol{\theta} - \mathbf{y}$ to be orthogonal to every column of $\mathbf{X}$ (that is what $\mathbf{X}^\top(\cdot) = \mathbf{0}$ says), placing $\mathbf{X}\boldsymbol{\theta}$ at the closest point of the column space to $\mathbf{y}$.

Numerics and cost

Forming $(\mathbf{X}^\top \mathbf{X})^{-1}$ is unstable when features are correlated ($\mathbf{X}^\top \mathbf{X}$ near-singular). In practice one uses the **SVD pseudoinverse** $\boldsymbol{\theta} = \mathbf{X}^+ \mathbf{y}$ (Module 2), which is stable and also handles the non-invertible case. The cost scales like $O(n^2 m + n^3)$ in the number of features n — fine for modest n , but for very many features or samples, gradient descent wins.

4.2 Gradient descent

When a closed form is too expensive or does not exist (as for the classifiers below), we minimise J iteratively by stepping downhill (Module 3):

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla J(\boldsymbol{\theta}), \quad \nabla J = \frac{2}{m} \mathbf{X}^\top(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) \text{ (for MSE)}. \quad (4.4)$$

Because the MSE surface is a convex bowl, gradient descent converges to the global minimum for a suitable learning rate η . Three variants trade noise against cost:

- **Batch** — use all m rows per step: exact gradient, but slow on big data.
- **Stochastic (SGD)** — one random row per step: cheap and noisy; the noise helps escape flat regions and (later) poor local minima, but it never settles exactly — decay η over time (a learning schedule).
- **Mini-batch** — a small batch per step: the practical default, smooth enough and hardware-efficient.

Key idea

Why scaling matters. The bowl’s shape is governed by the eigenvalues of $\mathbf{X}^\top \mathbf{X}$ (Module 2). Unscaled features make it a long narrow valley (ill-conditioned), and gradient descent zig-zags slowly across it. Standardising the features (Chapter 2) rounds the bowl and speeds convergence dramatically — the optimisation reason behind the preprocessing rule.

4.3 Polynomial regression, learning curves, and the trade-off

A linear model fits non-linear data once you add polynomial *features* ($x, x^2, x^3, \dots$): the model stays linear in the parameters while the fit curves. High degree overfits. **Learning curves** — training and validation error versus training-set size — diagnose which regime you are in: curves that converge to a high error signal **underfitting** (high bias); a persistent gap between low training error and high validation error signals **overfitting** (high variance). This is the Chapter 1 decomposition made visible, and it tells you whether to add capacity or to regularise.

4.4 Regularised linear models

Regularisation shrinks the parameters to curb variance. Add a penalty on $\boldsymbol{\theta}$ (excluding the bias) to the cost.

Ridge (ℓ_2). Penalise squared magnitude:

$$J_{\text{ridge}}(\boldsymbol{\theta}) = \frac{1}{m} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \alpha \|\boldsymbol{\theta}\|_2^2. \quad (4.5)$$

Setting the gradient to zero gives a closed form that is the normal equation with a ridge added to the diagonal:

$$\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (4.6)$$

The $\alpha \mathbf{I}$ term makes the matrix *always* invertible (it lifts every eigenvalue by $\alpha > 0$), curing the collinearity problem of the plain normal equation and shrinking all coefficients smoothly toward zero.

Lasso (ℓ_1). Penalise absolute magnitude:

$$J_{\text{lasso}}(\boldsymbol{\theta}) = \frac{1}{m} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \alpha \|\boldsymbol{\theta}\|_1. \quad (4.7)$$

The ℓ_1 penalty drives many coefficients *exactly to zero*, performing automatic **feature selection** — valuable when only a few of many biomarkers matter. Geometrically the ℓ_1 “ball” has corners on the axes, and the cost contours tend to touch it there (a zero coordinate); the smooth ℓ_2 ball has no corners, so Ridge shrinks but does not zero. Lasso has no closed form (the absolute value is not differentiable at 0); it is solved by sub-gradient or coordinate methods. **Elastic Net** blends the two penalties, and **early stopping** — halting gradient descent when validation error bottoms out — regularises implicitly by limiting how far the parameters travel.

4.5 Logistic regression

To classify, squash the linear score through the **logistic (sigmoid)** function into a probability:

$$\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x}), \quad \sigma(t) = \frac{1}{1 + e^{-t}}, \quad \sigma'(t) = \sigma(t)(1 - \sigma(t)). \quad (4.8)$$

We predict the positive class when $\hat{p} \geq 0.5$, i.e. when $\boldsymbol{\theta}^\top \mathbf{x} \geq 0$, so the **decision boundary is linear**. We cannot use MSE here (it is non-convex in $\boldsymbol{\theta}$ through σ); instead we use the **log loss** (binary cross-entropy), which is convex:

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]. \quad (4.9)$$

This is the negative log-likelihood of a Bernoulli model: it is large when the model confidently assigns the wrong class and near zero when it is confidently right.

Gradient of the log loss For a single example with $\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$, the derivative of the per-example loss $\ell = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$ is remarkably clean. Using $\frac{\partial \ell}{\partial \hat{p}} = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}} = \frac{\hat{p}-y}{\hat{p}(1-\hat{p})}$ and $\frac{\partial \hat{p}}{\partial \boldsymbol{\theta}} = \sigma'(\boldsymbol{\theta}^\top \mathbf{x}) \mathbf{x} = \hat{p}(1-\hat{p}) \mathbf{x}$, the chain rule collapses the $\hat{p}(1-\hat{p})$ factors:

$$\frac{\partial \ell}{\partial \boldsymbol{\theta}} = (\hat{p} - y) \mathbf{x} \implies \nabla J = \frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}).$$

The gradient is the same “error times input” form as linear regression — only $\hat{\mathbf{p}}$ is now the sigmoid output. There is no closed form (the equations are transcendental in $\boldsymbol{\theta}$), so we train by gradient descent. Adding an ℓ_2 penalty regularises exactly as in Ridge.

4.6 Softmax regression

For K mutually exclusive classes, give each class its own parameter vector $\boldsymbol{\theta}_k$, score every class, and normalise the scores into a probability distribution with the **softmax**:

$$s_k(\mathbf{x}) = \boldsymbol{\theta}_k^\top \mathbf{x}, \quad \hat{p}_k = \text{softmax}_k(\mathbf{s}) = \frac{e^{s_k}}{\sum_{j=1}^K e^{s_j}}. \quad (4.10)$$

The predicted class is the one with the highest probability. Training minimises the **categorical cross-entropy**, with the label one-hot encoded ($y_k = 1$ for the true class):

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \hat{p}_k^{(i)}. \quad (4.11)$$

Its gradient with respect to each class’s parameters keeps the same elegant form,

$$\nabla_{\boldsymbol{\theta}_k} J = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}, \quad (4.12)$$

so the identical gradient-descent machinery trains it. Softmax regression *is* the output layer of a neural-network classifier (Part II): logistic regression is its two-class special case.

Reading these models as one family

Linear, logistic and softmax regression are the same object seen three ways: a linear score $\boldsymbol{\theta}^\top \mathbf{x}$, fed through identity, sigmoid, or softmax, trained by minimising squared error or cross-entropy, with gradient $\propto \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y})$ in every case. Master this triplet and you have the template for every linear predictor — and the final layer of every deep network in Part II.

Exercises

1. Derive the normal equation from $\nabla J = \mathbf{0}$, then show that it forces the residual $\mathbf{X}\boldsymbol{\theta} - \mathbf{y}$ to be orthogonal to every column of $\mathbf{X}$. Identify this with the projection picture of Module 2 and Annex A.
2. Compare the cost of forming $(\mathbf{X}^\top \mathbf{X})^{-1}$ with solving the same system through the SVD, and state the condition-number argument that makes the second preferable.
3. The MSE cost has Hessian $\frac{2}{m} \mathbf{X}^\top \mathbf{X}$. Using the learning-rate law of Module 3, give the largest stable fixed step size, and say what the ratio $\lambda_{\max}/\lambda_{\min}$ predicts about the number of iterations.
4. Ridge and Lasso penalise $\|\boldsymbol{\theta}\|_2^2$ and $\|\boldsymbol{\theta}\|_1$. Which produces exactly-zero coefficients, and give the geometric reason in terms of the shape of the constraint region.
5. Verify the log-loss gradient: show the $\hat{p}(1-\hat{p})$ factors cancel and that $\partial \ell / \partial \boldsymbol{\theta} = (\hat{p} - y)\mathbf{x}$. Say in one sentence why linear, logistic and softmax regression share the same “error times input” gradient.
6. Show that softmax regression with $K = 2$ reduces to logistic regression, and identify the redundancy in the parameters that makes the reduction possible.
7. Sketch training and validation curves against training-set size for a high-bias model and for a high-variance model, and state the single visual cue that separates them.

5 Decision Trees

A decision tree asks a sequence of threshold questions about the features and reads off a prediction at the leaf it reaches. It needs no feature scaling, handles mixed data, and is fully **interpretable** — you can read the rules — which makes it attractive for clinical decision support.

5.1 Impurity

A good split makes its child nodes *pure* (dominated by one class). Two measures quantify impurity of a node with class proportions p_k :

$$\text{Gini } G = 1 - \sum_k p_k^2, \quad \text{entropy } H = - \sum_k p_k \log_2 p_k. \quad (5.1)$$

Both are zero for a pure node and maximal when classes are balanced; Gini is slightly cheaper and the usual default, entropy tends to produce marginally more balanced trees. They almost always agree.

5.2 The CART training objective

The CART algorithm grows the tree greedily, top-down. At each node it searches over every feature k and threshold t for the split that minimises the **impurity-weighted** cost of the two children:

$$J(k, t) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}, \quad (5.2)$$

where $m_{\text{left}}, m_{\text{right}}$ count the samples sent each way. Equivalently it *maximises* the **information gain** (parent impurity minus the weighted child impurity). It then recurses on each child and stops at a purity or size/depth limit. Finding the globally optimal tree is NP-hard, so this greedy heuristic — locally best split at each step — is what we use. For **regression** trees, replace impurity by the children’s weighted MSE; a leaf predicts the mean of its samples.

5.3 Regularisation and the catch

An unconstrained tree grows until every leaf is pure — it memorises the training set (high variance). We regularise with hyperparameters: maximum depth, minimum samples per leaf/split, or post-pruning. Even so, single trees are **unstable**: a small change in the data, or a rotation of the (axis-aligned) feature space, can yield a very different tree. This high variance is exactly the weakness that *ensembles* of trees are built to cancel — the subject of Chapter 6.

Exercises

1. A node holds 40 positives and 10 negatives. Compute its Gini impurity and its entropy. Which is larger, and does the ranking of candidate splits usually change between the two?
2. Show that Gini impurity is zero exactly for a pure node and maximal for a uniform one, for K classes.
3. CART is greedy. State what the globally optimal tree would require, and why greedy splitting is nevertheless the standard choice.
4. Explain why a tree is unchanged by any monotone rescaling of a single feature, yet can change completely when the data are rotated by 45° (Module 2). Which property of the split rule causes this?
5. Rank `max_depth`, `min_samples_leaf` and `max_leaf_nodes` by how directly each controls variance, and state which way each must move to reduce overfitting.

6 Ensemble Learning and Random Forests

Aggregating many imperfect models often beats any single one — the **wisdom of crowds**. The trick is to combine predictors whose errors are *decorrelated*, so that mistakes cancel on the vote.

6.1 Why voting works

Suppose n independent binary classifiers each are right with probability $p > 0.5$. By the law of large numbers the fraction voting correctly concentrates above 0.5, so the **majority vote** is right with probability $\sum_{k>n/2} \binom{n}{k} p^k (1-p)^{n-k} \rightarrow 1$ as $n \rightarrow \infty$. A crowd of weak, *independent*, better-than-chance voters becomes a strong one. The catch is “independent”: models trained on the same data make correlated errors, so the real work is **decorrelating** them. *Soft voting* (averaging predicted probabilities) usually beats *hard voting* (counting labels) because it uses each model’s confidence.

6.2 Bagging, pasting and out-of-bag

Bagging (bootstrap aggregating) trains each predictor on a different bootstrap sample — m rows drawn *with* replacement — then averages or votes. Resampling decorrelates the models and so reduces *variance* without raising bias; it is ideal for high-variance learners like trees. A neat bonus: each bootstrap omits, on average, a fraction

$$\left(1 - \frac{1}{m}\right)^m \xrightarrow{m \rightarrow \infty} e^{-1} \approx 0.368 \quad (6.1)$$

of the data — the **out-of-bag** examples — which serve as a free validation set for each predictor. (*Pasting* is the same idea sampling *without* replacement.)

6.3 Random forests

A **random forest** is bagging specialised to trees with one extra twist: at each split it considers only a *random subset* of features. This further decorrelates the trees (they cannot all seize on the same dominant feature), which lowers variance more. Averaging how much each feature reduces impurity across the forest yields a useful **feature-importance** ranking — handy for screening which biomarkers drive a prediction. *Extra-trees* randomise the split thresholds too, trading a little bias for still lower variance and faster training.

6.4 Boosting

Boosting takes the opposite tack: train predictors *sequentially*, each fixing its predecessor's mistakes, which mainly reduces *bias*.

AdaBoost. Maintain a weight on each training example; after each predictor, *raise* the weights of the examples it got wrong so the next predictor concentrates on them. A predictor with weighted error rate r_j earns a say in the final vote of

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}, \quad (6.2)$$

which is large when r_j is small (a good predictor) and zero at chance ($r_j = 0.5$). Misclassified examples then have their weights multiplied by $\exp(\alpha_j)$ and all weights are renormalised. The ensemble predicts the α -weighted vote. Boosting continues until a set number of predictors or until errors stop improving.

Gradient boosting. Instead of reweighting, fit each new predictor to the **residual errors** of the current ensemble. For squared loss the residual is simply $y - F(\mathbf{x})$, and more generally it is the negative gradient of the loss with respect to the current predictions (hence the name) — gradient descent in function space (Module 3). The ensemble is built additively,

$$F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \eta h_t(\mathbf{x}), \quad (6.3)$$

where h_t is the new tree fit to the residuals and the **learning rate** η (shrinkage) damps each step; smaller η with more trees usually generalises better. **XGBoost** and similar libraries are fast, regularised implementations that dominate tabular-data competitions and clinical risk models.

6.5 Stacking

Rather than a fixed vote, **stacking** trains a final model (a *blender*) to learn the best way to combine the base predictors' outputs, using held-out predictions so the blender does not simply echo the training fit.

Key idea

The bias–variance reading. Bagging and random forests attack *variance* (average many decorrelated low-bias trees); boosting attacks *bias* (sequentially correct a weak learner) but can overfit if run too long. This is the Chapter 1 decomposition choosing your ensemble strategy.

Exercises

1. A thousand *independent* classifiers are each right 51% of the time. Estimate the probability that their majority vote is right, and then state the assumption that fails for models trained on one dataset.

2. Explain how bagging reduces variance without raising bias, and why the out-of-bag samples give a validation estimate at no extra cost. Roughly what fraction of the training set is out-of-bag for each predictor?
3. A random forest subsamples features at every split. Say what this does to the bias and to the variance of the ensemble, and why the trade is usually favourable.
4. Describe AdaBoost and gradient boosting in one sentence each. Which of bagging and boosting cannot be parallelised across predictors, and why.
5. Gradient boosting fits each new tree to the residuals. Explain the sense in which this is gradient descent in function space, using the loss gradient of Module 3.
6. Impurity-based feature importances are reported for a forest with two strongly correlated features. Say what happens to their importances and what you would use instead.

7 Dimensionality Reduction

High-dimensional data is hard — to store, to visualise, and to learn from. Reducing the number of features, while keeping what matters, speeds training, fights overfitting, and lets us *see* the data.

7.1 The curse of dimensionality

Intuition built in two or three dimensions fails in many. As dimension grows, data becomes **sparse**: in a unit hypercube of dimension n , a sub-cube covering 10% of each axis holds only 0.1^n of the volume, so almost all the volume sits near the boundary, and random points sit far apart. Pairwise distances **concentrate** (nearest and farthest neighbours become almost equidistant), which blunts distance-based methods, and the data needed to densely cover the space grows *exponentially*. The escape: real data rarely fills its space — it lies near a much lower-dimensional **subspace** (use projection) or a curved **manifold** (use manifold learning).

7.2 PCA via the SVD

Principal-component analysis finds the orthonormal directions of greatest variance and projects onto the top few. Centre the data matrix $\mathbf{X}$ (subtract each feature's mean) and take its singular value decomposition (Module 2), $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. The columns of $\mathbf{V}$ — the right singular vectors — are the **principal components**, ordered by the singular values $\sigma_1 \geq \sigma_2 \geq \dots$. Equivalently they are the eigenvectors of the covariance matrix $\frac{1}{m}\mathbf{X}^\top\mathbf{X}$, which is symmetric, so by the spectral theorem they are orthonormal. Projecting onto the first d components,

$$\mathbf{X}_d = \mathbf{X}\mathbf{V}_d, \tag{7.1}$$

gives the lower-dimensional coordinates; multiplying back by $\mathbf{V}_d^\top$ reconstructs an approximation in the original space.

Why these axes? Two equal views PCA's top- d subspace simultaneously **maximises the retained variance** and **minimises the squared reconstruction error**. These are the same problem: by Pythagoras, total variance = retained variance + lost (reconstruction) variance is constant, so maximising one minimises the other. The SVD's optimal low-rank approximation (Eckart–Young, Module 2) delivers exactly this subspace, with the σ_i^2 giving the variance along each axis.

The **explained-variance ratio** of component i is $\sigma_i^2 / \sum_j \sigma_j^2$; choose d to retain a target (say 95%) of the variance, or to drop to 2–3 for plotting. Reducing 30 assay features to two principal components to see malignant and benign samples separate (as in Module 2) is PCA doing exactly this.

7.3 Variants and manifold methods

Randomised PCA approximates the top components quickly for large data; **incremental PCA** streams data in batches when it will not fit in memory; and **kernel PCA** applies the kernel trick of the online SVM appendix (Appendix A) to find *non-linear* principal components. When the data lies on a curved manifold, **manifold learning** unrolls it: **LLE** preserves local linear relationships, and **t-SNE** (and UMAP) preserve local neighbourhoods to make striking 2-D visualisations of clusters — excellent for inspecting cell populations or expression data, though meant for visualisation, not as features for a downstream model.

Exercises

1. In a unit hypercube of dimension n , compute the fraction of the volume lying within 0.01 of the boundary for $n = 2, 10$ and 100 . State what this implies for nearest-neighbour methods.
2. Explain what goes wrong if the data matrix is not centred before taking the SVD, in terms of what the first principal component then measures.
3. Singular values are $\sigma = (12, 7, 3, 1.5, 0.6)$. Compute the explained-variance ratios and the smallest d retaining 95% of the variance.
4. Prove that maximising retained variance and minimising squared reconstruction error are the same problem, using the Pythagorean identity named in the box above.
5. PCA is run on features measured in millimetres, kilopascals and counts. Predict which component dominates and why, and give the one-line fix.
6. You obtain a beautiful two-dimensional t-SNE plot. State two things it is *not* evidence for, in particular about distances between clusters.

8 Unsupervised Learning Techniques

Without labels, we look for *structure*: groups (clustering), densities, and outliers. These methods organise data and flag the unusual — a natural fit for exploratory biomedical analysis and anomaly detection.

8.1 k -means

k -means partitions the data into k clusters by minimising the **inertia**, the within-cluster sum of squared distances to the cluster centre:

$$J = \sum_{i=1}^m \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|^2, \quad (8.1)$$

where $c(i)$ is point i 's assigned cluster and $\boldsymbol{\mu}_k$ its centroid. Lloyd's algorithm alternates two steps:

1. **Assign** each point to the nearest centroid;
2. **Update** each centroid to the mean of its assigned points.

Each step can only lower J (the assignment minimises it over c with $\boldsymbol{\mu}$ fixed; the mean minimises it over $\boldsymbol{\mu}$ with c fixed), so this is **coordinate descent** on the inertia and converges — but only to a *local* minimum that depends on the initial centroids. **k-means++** seeds centroids spread far apart, greatly improving the result; one also restarts several times and keeps the lowest-inertia run. Choose k by the **elbow** of inertia-vs- k or by the **silhouette** score. The method's assumptions — roughly spherical, similar-sized clusters, and a known k — are also its limitations.

8.2 DBSCAN

DBSCAN clusters by *density* instead of distance to a centre. A point is a **core point** if at least `min_samples` points lie within radius ε ; core points that are within ε of each other join into a cluster, border points attach to a nearby core, and points in no dense region are labelled **noise**. It needs no k , finds clusters of *arbitrary shape*, and — because it explicitly marks low-density points — doubles as an **anomaly detector**. Its two hyperparameters are ε and `min_samples`; it struggles when clusters have very different densities.

8.3 Gaussian mixtures and EM

A **Gaussian mixture model** assumes the data are drawn from a blend of K Gaussians,

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \sum_k \pi_k = 1, \quad (8.2)$$

giving **soft** cluster assignments (a probability of belonging to each component) and a full *density* estimate. It is fit by the **expectation–maximisation (EM)** algorithm, which alternates:

- **E-step** — with the parameters fixed, compute each point's **responsibility** (posterior membership)

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)};$$

- **M-step** — with responsibilities fixed, re-estimate each component as a responsibility-weighted average: $\boldsymbol{\mu}_k = \frac{\sum_i \gamma_{ik} \mathbf{x}_i}{\sum_i \gamma_{ik}}$, the covariance $\boldsymbol{\Sigma}_k$ as the weighted scatter, and the weight $\pi_k = \frac{1}{m} \sum_i \gamma_{ik}$.

Each round increases the data likelihood and converges to a local maximum. EM is the soft, probabilistic generalisation of k -means — indeed k -means is the limit of a GMM with hard assignments and equal spherical covariances. The fitted density flags low-probability points as **anomalies**, and the number of components is chosen by an information criterion (**BIC/AIC**) that penalises complexity.

Part I complete — the through-line

Across the eight chapters of Part I one pattern recurs: choose a model, write the cost that scores it, and optimise. Linear, logistic and softmax regression minimise squared error or cross-entropy by gradient descent; SVMs maximise a margin (minimise an ℓ_2 -plus-hinge cost) and kernelise the dot product; trees greedily minimise impurity; ensembles combine many such models to trade bias against variance; PCA minimises reconstruction error through the SVD; and k -means and GMMs minimise inertia / maximise likelihood by (hard or soft) coordinate ascent. The linear algebra of Module 2 (projection, the SVD, the spectral theorem) and the calculus of Module 3 (the gradient, gradient descent) are the two engines underneath all of it. **Part II** keeps the same recipe — model, loss, gradient descent — and lets the model become a deep neural network.

Exercises

1. Show that each of Lloyd's two steps can only lower the inertia J , and conclude that the algorithm converges. Explain why convergence does not mean the optimum was found.
2. Inertia decreases monotonically with k . Explain why it therefore cannot be used alone to choose k , and describe what the elbow and the silhouette score each add.
3. Sketch a dataset of two elongated, unequally sized clusters on which k -means fails, say which assumption it violates, and name a method that handles it.
4. For DBSCAN, describe what happens to the clustering as $\varepsilon \rightarrow 0$ and as ε grows large, and how `min_samples` interacts with noise labelling.
5. A Gaussian mixture with spherical, equal, shrinking covariances recovers k -means. Explain the limit, and state what a full covariance buys you.
6. EM increases the likelihood at every iteration. Say what this guarantees, what it does not, and what practitioners therefore do about initialisation.

Appendix A Support Vector Machines

In the *PyTorch* edition, support vector machines are presented as an appendix rather than as a numbered chapter of Part I; the notebook is `Appendix_C_support_vector_machines.ipynb`. The theory is collected here for completeness — SVMs remain a strong choice for the high-dimensional, medium-sized datasets common in biomedical assays.

A support vector machine classifies by fitting the **widest possible street** between the classes. Of all separating hyperplanes, it picks the one whose margin to the nearest points is largest — a principled bet that a wide margin generalises well. The nearest points, which alone determine the boundary, are the **support vectors**.

A.1 The margin

For a linear boundary $\mathbf{w}^\top \mathbf{x} + b = 0$, the signed distance from a point to it is $(\mathbf{w}^\top \mathbf{x} + b)/\|\mathbf{w}\|$. Fix the scale so that the closest points satisfy $\mathbf{w}^\top \mathbf{x} + b = \pm 1$; the two margin edges are then $\mathbf{w}^\top \mathbf{x} + b = \pm 1$ and the street width is

$$\text{margin} = \frac{2}{\|\mathbf{w}\|}. \quad (\text{A.1})$$

Maximising the margin therefore means *minimising* $\|\mathbf{w}\|$. With labels $y_i \in \{-1, +1\}$, requiring every point on the correct side of its margin gives the **hard-margin** problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i. \quad (\text{A.2})$$

A.2 Soft margin and the hinge loss

Real data are noisy and rarely linearly separable, so we allow violations through slack variables $\xi_i \geq 0$ and pay for them:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (\text{A.3})$$

The hyperparameter C trades margin width against violations: small C = wide margin, more violations (more regularisation, higher bias); large C = narrow margin, fewer violations (higher

variance). Eliminating the slacks rewrites this as an unconstrained **hinge-loss** objective,

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)), \quad (\text{A.4})$$

which reads exactly like a regularised linear model: an ℓ_2 penalty plus a loss that is zero once a point is correctly classified *beyond* the margin and grows linearly inside it.

A.3 The dual and the kernel trick

Introducing Lagrange multipliers $\alpha_i \geq 0$ for the constraints and eliminating $\mathbf{w}, b$ yields the **dual** problem

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0, \quad (\text{A.5})$$

with the recovered weights $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ — a combination of only the support vectors (those with $\alpha_i > 0$). The decisive observation: the data appear *only* through dot products $\mathbf{x}_i^\top \mathbf{x}_j$. We may therefore replace each dot product by a **kernel** $K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i)^\top \varphi(\mathbf{x}_j)$, the dot product in some high-dimensional feature space, *without ever computing* φ :

$$\text{linear } K = \mathbf{x}^\top \mathbf{x}', \quad \text{polynomial } K = (\gamma \mathbf{x}^\top \mathbf{x}' + r)^d, \quad \text{RBF } K = e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|^2}. \quad (\text{A.6})$$

This **kernel trick** buys non-linear decision boundaries at linear cost; the Gaussian RBF kernel corresponds to an *infinite*-dimensional feature space, yet each evaluation is one cheap formula. SVMs also do regression (fit the most points *inside* an ε -tube rather than outside a margin).

Key idea

Practical notes. SVMs are scale-sensitive (the margin is a Euclidean distance) — always standardise. The key hyperparameters are C (regularisation) and, for RBF, γ (how far a single point’s influence reaches). SVMs shine in high-dimensional, medium-sized problems — common in biomedical assays — but scale poorly to very large m , where ensembles or linear models are preferred.

Exercises

1. Starting from the signed distance $(\mathbf{w}^\top \mathbf{x} + b)/\|\mathbf{w}\|$ and the ± 1 scaling, derive margin = $2/\|\mathbf{w}\|$, and explain why maximising the margin becomes minimising $\frac{1}{2}\|\mathbf{w}\|^2$.
2. Describe the effect of a very small and a very large C on margin width, the number of violations, and the bias–variance position of the model.
3. Plot the hinge loss and the logistic loss against the margin $y(\mathbf{w}^\top \mathbf{x} + b)$ on the same axes. Which is exactly zero beyond the margin, and what does that imply about which points determine the solution?
4. State why the dual formulation never needs the feature map φ explicitly, and what the RBF kernel corresponds to in terms of the dimension of that implicit space.
5. Explain why SVMs demand scaled features, using the objective rather than folklore — and connect it to the scaling discussion of Module 1.
6. The dual has one variable per training sample. State the consequence for very large m , and name the linear-SVM alternative you would reach for.